

Claude Code ↔ GitHub Copilot Agent Mode : Points de convergence

Objet : Identifier ce qui est directement transférable et ce qui est fonctionnellement similaire entre Claude Code et GitHub Copilot (agent mode / coding agent / CLI), en termes de features, workflows et pratiques de prompt engineering.

Date : Mars 2026 — **Sources :** documentation officielle GitHub Docs, Anthropic Docs, GitHub Blog, VS Code Docs.

1. Vue d'ensemble architecturale

Les deux outils ont convergé vers une architecture quasi-identique en couches :

Couche	Claude Code	Copilot Agent Mode
Boucle agentique	Terminal REPL autonome : analyse codebase → édite fichiers → exécute commandes → auto-corrige en boucle	Agent mode IDE (VS Code, JetBrains, Xcode, Eclipse) : même boucle autonome — analyse, édite, exécute, auto-corrige
Agent asynchrone	<code>claude --background</code> / Agent Teams (preview)	Copilot Coding Agent : assigner une issue GitHub → PR automatique en arrière-plan via GitHub Actions
CLI dédié	<code>claude</code> (natif terminal)	<code>copilot</code> CLI (GA depuis février 2026) — même paradigme terminal-natif
Instructions persistantes	<code>CLAUDE.md</code> (hiérarchie : global → projet → dossier)	<code>.github/copilot-instructions.md</code> + <code>*.instructions.md</code> + <code>AGENTS.md</code> (hiérarchie similaire)
Extensions	MCP servers, Hooks, Skills, Plugins	MCP servers, Hooks, Skills, Custom Agents, Plugins
Sous-agents	Subagents (Explore, Plan, custom)	Subagents + Custom Agents (<code>.agent.md</code>) avec handoffs
Gestion de contexte	Auto-compaction à ~95%, <code>/compact</code> , extended thinking	Auto-compaction à ~95%, <code>/compact</code> (CLI), plan mode

Verdict : L'architecture est convergente. Un praticien Claude Code retrouvera un modèle mental familier.

2. Features transférables (directement)

Ces features existent des deux côtés avec un fonctionnement suffisamment proche pour que les habitudes soient transférées quasi telles quelles.

2.1 Instructions projet (CLAUDE.md ↔ copilot-instructions.md)

Aspect	Claude Code	Copilot
Fichier principal	CLAUDE . md à la racine du projet	.github/copilot-instructions . md à la racine du repo
Instructions imbriquées par dossier	CLAUDE . md dans sous-dossiers (chargement lazy)	*.instructions . md avec frontmatter applyTo pour ciblage par glob path
Instructions globales (user)	~/ . claude / CLAUDE . md	Instructions personnelles dans les settings Copilot
Instructions organisation	Non applicable directement	Instructions au niveau org via settings GitHub
Format	Markdown, langage naturel	Markdown, langage naturel
Interopérabilité	—	VS Code lit nativement les fichiers CLAUDE . md (setting chat . useClaudeMdFile). Le Copilot Coding Agent supporte aussi CLAUDE . md.

Ce qui se transfère directement : la pratique de structurer un fichier d'instructions avec conventions de code, commandes de build/test, patterns à suivre/éviter. Le contenu d'un CLAUDE . md peut être copié quasi tel quel dans un copilot-instructions . md.

Bonnes pratiques partagées :

- Instructions courtes, atomiques, une règle par paragraphe
- Inclure le *pourquoi* derrière chaque règle (aide le modèle dans les cas limites)
- Montrer des exemples concrets de code (pattern préféré vs pattern à éviter)
- Ne pas dupliquer ce que les linters/formatters gèrent déjà
- Versionner le fichier d'instructions dans le repo

2.2 Support MCP (Model Context Protocol)

Aspect	Claude Code	Copilot
Configuration	<code>.mcp.json</code> à la racine ou <code>settings.json</code>	<code>mcp.json</code> dans <code>.vscode/</code> (IDE) ou settings du repo (coding agent)
Ajout interactif	<code>claude mcp add</code>	<code>/mcp add</code> (CLI) ou GUI dans VS Code
Serveurs locaux (stdio)	Supporté	Supporté (mappé depuis le type <code>stdio</code> de Claude Code)
Serveurs distants (HTTP/SSE)	Supporté	Supporté
MCP dans l'agent asynchrone	Via config projet	Via settings repo GitHub (coding agent) — secrets préfixés <code>COPILOT_MCP_</code>
Contrôle des outils	Sélection par serveur	Sélection par outil individuel dans l'IDE + allowlists dans les custom agents

Ce qui se transfère directement : toute configuration MCP existante. Le Copilot coding agent reconnaît le format `stdio` utilisé par Claude Code et le mappe automatiquement vers son type `local`. Les serveurs MCP tiers (GitHub MCP, Playwright, etc.) fonctionnent des deux côtés.

2.3 Hooks

Aspect	Claude Code	Copilot
Emplacement config	<code>settings.json</code> (Claude Code)	<code>.github/hooks/*.json</code> (repo)
Événements principaux	<code>PreToolUse</code> , <code>PostToolUse</code> , <code>Stop</code> , <code>SessionStart</code> , <code>SessionEnd</code> , <code>PermissionRequest</code>	<code>preToolUse</code> , <code>postToolUse</code> , <code>sessionStart</code> , <code>sessionEnd</code> , <code>userPromptSubmitted</code> , <code>agentStop</code> , <code>subagentStop</code> , <code>errorOccurred</code>
Blocage d'actions	Exit code 2 bloque l'action	<code>preToolUse</code> peut approuver ou refuser les appels d'outils
Types de hooks	Command (shell), HTTP, LLM prompt	Command (shell)
Périmètre	Projet ou user-level	Repo (coding agent + CLI)

Ce qui se transfère directement : le concept et la pratique — lint automatique après écriture de fichier, blocage de commandes destructrices, notifications post-tâche. La logique des scripts shell est portable. Les matchers changent de syntaxe mais le modèle mental est identique : événement + filtre + action.

Différence notable : Claude Code supporte des hooks de type HTTP et LLM prompt (un prompt envoyé à un modèle comme hook), Copilot se limite actuellement aux commandes shell.

2.4 Skills (agent skills)

Aspect	Claude Code	Copilot
Format	Dossier avec <code>SKILL.md</code> + ressources (scripts, templates)	Dossier avec <code>SKILL.md</code> + ressources (scripts, templates)
Emplacement projet	<code>.claude/skills/</code>	<code>.copilot/skills/</code> ou <code>.claude/skills/</code> (Copilot lit les deux)
Emplacement user	<code>~/.claude/skills/</code>	<code>~/.copilot/skills/</code>
Déclenchement	Automatique quand le modèle juge le skill pertinent	Automatique quand le modèle juge le skill pertinent
Invocation manuelle	<code>/skill-name</code> (slash command)	<code>/skill-name</code> (slash command)

Ce qui se transfère directement : les skills Claude Code sont lus nativement par Copilot. Les dossiers `.claude/skills/` sont découverts automatiquement par Copilot coding agent et Copilot CLI. Aucune migration nécessaire.

2.5 Gestion du contexte

Aspect	Claude Code	Copilot
Auto-compaction	À ~95% de la fenêtre de contexte	À ~95% de la fenêtre de contexte
Compaction manuelle	<code>/compact</code>	<code>/compact</code> (CLI)
Sous-agents pour isoler le contexte	Subagent Explore (read-only, rapide)	Subagent Explore (CLI, codebase analysis)
Mode plan	<code>Shift+Tab</code> pour basculer en plan mode	<code>Shift+Tab</code> (CLI) ou <code>/plan</code>
Nettoyage	<code>/clear</code>	<code>/clear</code> ou <code>/new</code>

Ce qui se transfère directement : les reflexes de gestion de contexte — utiliser `/compact` avant saturation, déléguer l'exploration à un sous-agent dédié, nettoyer entre tâches non liées, planifier avant d'implémenter.

2.6 Sous-agents et agents spécialisés

Aspect	Claude Code	Copilot
Format de définition	<code>.claude/agents/*.md</code> avec frontmatter YAML	<code>.github/agents/*.agent.md</code> avec frontmatter YAML
Built-in	Explore (read-only), Plan (recherche pour planification)	Explore (codebase analysis), Task (build/test)
Restriction d'outils	<code>tools</code> (allowlist) ou <code>disallowedTools</code> (denylist)	<code>tools</code> (allowlist) dans le profil agent
Handoffs (chaînage)	Via prompt orchestration manuelle	Handoffs natifs : un agent termine → bouton pour passer au suivant avec contexte
Partage organisation	Via plugins	Via <code>.github-private</code> repo au niveau org/enterprise
Modèle par agent	Configurable (<code>model</code> dans frontmatter)	Configurable (<code>model</code> dans frontmatter)

Ce qui se transfère : la pratique de créer des agents spécialisés (review, docs, security, migration). La structure est similaire : un fichier markdown avec YAML frontmatter définissant le persona, les outils autorisés, et les instructions.

Valeur ajoutée Copilot : les handoffs natifs entre agents permettent un workflow guidé (Plan → Implement → Review) avec passage de contexte automatique — une feature que Claude Code n'a pas nativement (nécessite orchestration manuelle).

3. Features similaires (même intention, implémentation différente)

3.1 Prompt files réutilisables

Claude Code	Copilot
Slash commands : fichiers <code>.md</code> dans <code>.claude/commands/</code> ou <code>.claude/skills/</code>	Prompt files : fichiers <code>.prompt.md</code> dans <code>.github/prompts/</code>
Invoqués via <code>/nom-commande</code>	Invoqués via <code>/nom-prompt</code> dans le chat

Claude Code	Copilot
Markdown simple	Markdown avec frontmatter YAML optionnel (agent), (model), (tools), (description)

Similitude : même concept — encapsuler un workflow répétitif dans un fichier invocable à la demande. Les prompt files Copilot ajoutent le frontmatter YAML pour spécifier le modèle et les outils, ce qui est plus explicite.

3.2 Agent asynchrone (background)

Claude Code	Copilot
<code>claude --background</code> ou Agent Teams (research preview)	Copilot Coding Agent : assigner une issue → PR
Exécution locale (ou remote via Copilot SDK)	Exécution dans un environnement GitHub Actions sécurisé
Pas de PR automatique	Crée une draft PR, request review, itère sur les commentaires
Pas d'intégration native avec issue tracker	Déclenchement depuis GitHub Issues, Jira, Linear, Slack, Teams

Similitude : délégation de tâches en arrière-plan pendant que le développeur travaille sur autre chose. Mais l'implémentation est fondamentalement différente : Claude Code reste local, Copilot Coding Agent est une CI/CD-native workflow.

3.3 Permissions et sécurité

Claude Code	Copilot
Système allow/deny granulaire dans <code>settings.json</code>	Policies organisationnelles via settings GitHub
<code>--allowedTools</code> , <code>--dangerouslySkipPermissions</code>	<code>--allow-all</code> / <code>--yolo</code> (CLI)
Hooks <code>PermissionRequest</code> pour automatiser	Hooks <code>preToolUse</code> pour approuver/refuser
Permissions par subagent	Permissions par custom agent (tools allowlist)

Similitude : même philosophie — contrôle granulaire des outils, possibilité de bypass total en mode confiance, hooks pour enforcement déterministe.

3.4 Mémoire inter-sessions

Claude Code	Copilot
<code>CLAUDE.md</code> comme mémoire explicite + <code>#</code> pour quick memory	Agentic Memory : Copilot déduit et stocke des infos utiles sur le repo
Mémoire manuelle, contrôlée par le développeur	Mémoire automatique (Copilot Pro/Pro+)
<code>/memory</code> pour éditer	Pas d'édition directe de la mémoire

Similitude : persistance de connaissances entre sessions. Mais l'approche est inversée — Claude Code est explicite et piloté par le dev, Copilot est implicite et piloté par l'IA.

3.5 Choix de modèle

Claude Code	Copilot
<code>/model</code> pour switcher (Opus, Sonnet, Haiku)	<code>/model</code> pour switcher (GPT-4o, GPT-5, o3, Claude Sonnet/Opus, Gemini, Codex)
Modèle configurable par subagent	Modèle configurable par custom agent et par tâche (coding agent)
Anthropic uniquement	Multi-fournisseur (OpenAI, Anthropic, Google, xAI)

Transférable : le réflexe de choisir le modèle selon la tâche — modèle lourd pour raisonnement complexe, modèle léger pour exploration rapide.

4. Pratiques de prompt engineering transférables

4.1 Structuration du prompt

Les bonnes pratiques sont identiques des deux côtés :

- **Contexte d'abord** : décrire le problème, le stack technique, les contraintes avant de demander une action
- **Spécificité** : éviter "corrige ce bug" → préférer "le endpoint `/api/users` retourne 500 de façon intermittente, cherche dans le codebase et les logs pour identifier la cause racine"
- **Exemples** : montrer le format de sortie attendu avec des exemples d'input/output concrets

- **Décomposition** : pour les tâches complexes, utiliser le plan mode (les deux outils le supportent) puis implémenter par étapes

4.2 Workflow agentic en phases

Les deux outils bénéficient d'un workflow en phases distinctes plutôt qu'un prompt monolithique :

1. **Recherche** → sous-agent Explore / codebase scan
2. **Planification** → plan mode, review humain du plan
3. **Implémentation** → agent mode, itération automatique
4. **Validation** → exécution des tests, lint, review

Cette pratique est directement transférable. Les deux outils supportent `/plan` ou `Shift+Tab` pour séparer planification et exécution.

4.3 Instructions comme investissement

La pratique la plus rentable — investir dans des fichiers d'instructions de qualité — est identique :

- Un fichier d'instructions bien écrit évite de répéter le même contexte à chaque session
- Les instructions devraient être itératives : commencer simple, ajouter des règles quand l'agent fait des erreurs
- Les instructions générales vont dans le fichier principal, les instructions spécifiques à une tâche dans des skills/prompts dédiés
- Ne pas mélanger les deux :  dupliquer des règles entre instructions et prompts —  garder les instructions générales, les prompts spécifiques

4.4 Anti-patterns communs

Ces erreurs sont documentées des deux côtés :

- **Prompt trop vague** : "tu es un assistant utile" ne fonctionne ni avec Claude Code ni avec Copilot
 - **Instructions trop longues** : au-delà d'un certain volume, les deux outils commencent à ignorer des instructions (limites de contexte)
 - **Conflits entre instructions** : si `copilot-instructions.md` dit "use camelCase" et un prompt file dit "use snake_case", le comportement est non-déterministe dans les deux cas
 - **Pas de tests dans les instructions** : les deux outils performant mieux quand les instructions incluent "lance les tests après chaque modification"
-

5. Matrice de correspondance rapide

Concept / Feature	Claude Code	Copilot équivalent
Instructions projet	<code>CLAUDE.md</code>	<code>.github/copilot-instructions.md</code>
Instructions par dossier	<code>CLAUDE.md</code> imbriqués	<code>*.instructions.md</code> avec <code>applyTo</code>
Instructions agent	Dans subagent <code>.md</code>	<code>AGENTS.md</code> ou <code>.agent.md</code>
Slash commands	<code>.claude/commands/*.md</code>	<code>.github/prompts/*.prompt.md</code>
Skills	<code>.claude/skills/*/SKILL.md</code>	<code>.copilot/skills/*/SKILL.md</code> (ou <code>.claude/skills/</code>)
Sous-agents	<code>.claude/agents/*.md</code>	<code>.github/agents/*.agent.md</code>
MCP servers	<code>.mcp.json</code>	<code>.vscode/mcp.json</code> (IDE) ou settings repo (coding agent)
Hooks	<code>settings.json</code> hooks	<code>.github/hooks/*.json</code>
Plan mode	<code>Shift+Tab</code>	<code>Shift+Tab</code> (CLI) / <code>/plan</code>
Auto-compaction	Automatique ~95%	Automatique ~95%
Quick memory	<code># note rapide</code>	Agentic Memory (automatique)
Review	<code>/review</code>	<code>/review</code>
Plugins	<code>claude plugin install</code>	<code>copilot plugin install</code>
Extended thinking	<code>Tab</code> pour toggle	<code>Ctrl+T</code> pour toggle (CLI)

6. Ce qui n'a pas d'équivalent direct

Exclusif à Claude Code

- Agent Teams** : orchestration multi-agents en parallèle avec communication (research preview)

- **Hooks de type HTTP et LLM prompt** : seul Claude Code permet de déclencher un appel API ou un prompt LLM comme hook
- **# quick memory** : ajout de notes au contexte en une ligne, sans fichier
- **Esc-Esc rewind** : revenir en arrière sur les modifications fichier à n'importe quel snapshot de la session

Exclusif à Copilot

- **Handoffs natifs entre agents** : chaînage guidé Plan → Implement → Review avec passage de contexte automatique
 - **Coding Agent cloud-native** : exécution dans GitHub Actions avec draft PR, code review intégrée, secret scanning, dependency scanning gratuits
 - **Multi-provider** : accès à Claude, GPT, Gemini, Codex, Grok dans le même outil
 - **Copilot Spaces** : organisation de contexte (code, docs, specs) dans des espaces dédiés pour ancrer les réponses
 - **Chat modes** : personas pré-configurées (`.chatmode.md`) avec instructions et outils spécifiques
 - **/fleet** : exécution parallèle de la même tâche sur plusieurs modèles pour converger sur un résultat
-

7. Recommandation de transition

Si tu passes de Claude Code à Copilot agent mode en entreprise :

1. **Jour 1** — Copier ton `CLAUDE.md` dans `.github/copilot-instructions.md`. Le contenu est portable tel quel.
 2. **Jour 1** — Tes skills dans `.claude/skills/` sont déjà lus par Copilot. Rien à migrer.
 3. **Semaine 1** — Adapter tes hooks : réécrire les entrées `settings.json` en fichiers `.github/hooks/*.json`. Seule la syntaxe de configuration change, pas la logique.
 4. **Semaine 1** — Convertir tes subagents (`.claude/agents/*.md`) en custom agents Copilot (`.github/agents/*.agent.md`). Profiter des handoffs natifs pour structurer les workflows.
 5. **Semaine 2** — Convertir tes slash commands en prompt files (`.github/prompts/*.prompt.md`) et exploiter le frontmatter YAML pour spécifier modèle et outils.
 6. **En continu** — Exploiter les features exclusives Copilot : coding agent asynchrone, multi-modèles, Spaces.
-

